Lost in the Middle — Test Runs and Limitations

11 November 2026

Code repository: https://github.com/coolprash06/Lost-In-the-Middle

Overview

We built a Streamlit app that re-runs the key-value retrieval experiment from Liu et al., "Lost in the Middle: How Language Models Use Long Contexts" (TACL 2024), on current models through the Groq API. Across every model and setting we could fit within the free tier, we did not reproduce the U-shaped curve.

How one test works:

1. The app builds a JSON object of N key-value pairs, where every key and value is a random UUID. Because they are random, the model cannot know them from training and must find the answer in the context.
2. One pair is the target. It is placed at one of five positions: First, 25%, Middle, 75% or Last.
3. The model is asked: "What is the value for key <target key>? Answer with only the value." Temperature is 0.
4. The answer counts as correct if the target's value appears in the reply.
5. This repeats T times per position, and the app plots accuracy by position. Every run is saved as a JSON file in the results/ folder.

The paper's finding is that accuracy is highest at the start and end of the context and lowest in the middle, forming a U-shape. Our app tests whether today's models still show that pattern.

Test runs

We ran every test on 11 November 2026 with T = 1 trial per position. Five runs scored 100% at every position, one scored 0% at every position, and three attempts failed with an error before producing a result.

#	Model	N (pairs)	Settings	Result	Notes
1	llama-3.1-8b-instant	75	Default	Error 404	Model no longer offered by Groq
2	openai/gpt-oss-20b	75	Reasoning effort low	100% at all 5 positions	About 4,700 tokens per request
3	openai/gpt-oss-20b	200	Reasoning effort low	Error 413	Request was 10,506 tokens; limit is 8,000 per minute
4	openai/gpt-oss-20b	140	Reasoning effort low	Error 413	Request was 8,092 tokens; just over the limit
5	openai/gpt-oss-20b	125	Reasoning effort low	100% at all 5 positions	Largest context that fit, about 7,300 tokens

#	Model	N (pairs)	Settings	Result	Notes
6	openai/gpt-oss-20b	50	Reasoning effort low	100% at all 5 positions	Run by mistake before switching model
7	allam-2-7b	50	Non-reasoning	0% at all 5 positions	Could not copy a UUID correctly; one reply was 63 zeros
8	qwen/qwen3.8-27b	100	Thinking off	100% at all 5 positions	Closest match to the paper's setup
9	qwen/qwen3.8-27b	100	Thinking off, hard mode, 10 look-alike keys	100% at all 5 positions	0 answers fooled by a look-alike key

In hard mode, 10 distractor keys are copies of the target key with 2 or 3 characters changed. They are spread randomly across the list, so the model has to read the exact key carefully at every position.

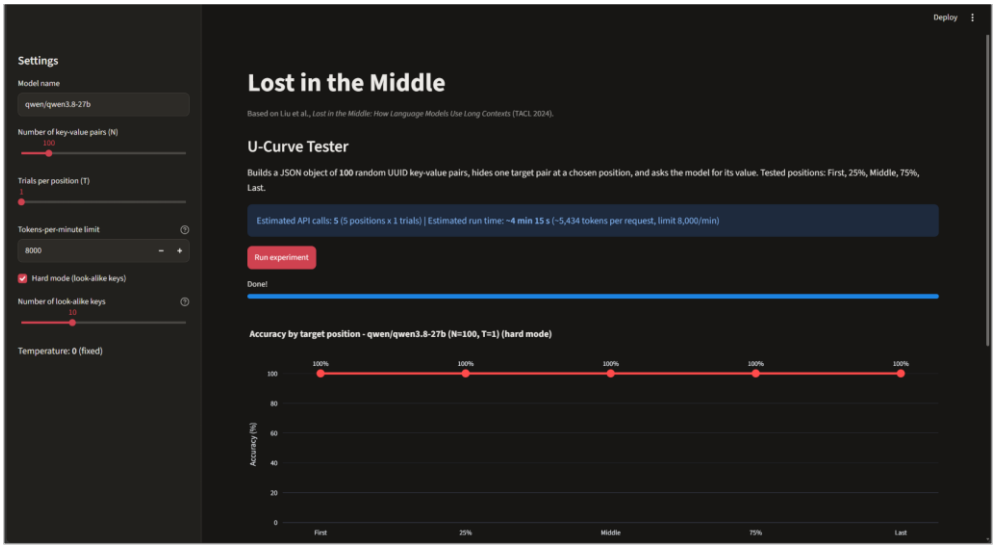


Figure 1: Run 9 — qwen/qwen3.8-27b, hard mode, N = 100. Accuracy is 100% at every position.



Figure 2: Run 9 results table — 0 answers fooled by look-alike keys.

What the results show

No model showed the U-shape: every model scored either 100% or 0% at all five positions, so position made no difference.

- **Mid-size and large models no longer get lost at these lengths.** gpt-oss-20b (a reasoning model) and Qwen 27B (with thinking turned off) found the target at every position, up to about 7,300 tokens of context. Qwen also ignored all 10 look-alike keys in hard mode.
- **A very small model fails everywhere, not just in the middle.** allam-2-7b could not reliably copy a 36-character UUID. It dropped characters, copied JSON symbols into its answer, and once got stuck repeating zeros. Its failures are about copying, not position.
- **The useful range is narrow.** The U-curve appears in models that can do the task but struggle with long input. Within our token budget, we found no model on Groq's free tier in that range.

One likely reason is that newer models are trained on much longer contexts than the 2023 models in the paper. At our context sizes (2,800 to 7,300 tokens), this task appears to be too easy for them.

Limitations

The biggest limit is Groq's free tier: 8,000 tokens per minute, which caps one request at about 7,300 tokens of context. The paper tested longer contexts on older models, so our results cover only the short end of its range.

Token limits

Limit	Value	Effect on our tests
Tokens per minute (gpt-oss-20b, free tier)	8,000	A single request must fit under it, or Groq rejects it with error 413. Waiting does not help.
What counts toward the limit	Prompt tokens + the reserved reply length	gpt-oss reserves 1,024 tokens for its reasoning and answer, leaving less room for the context.
Largest N that fit (gpt-oss-20b)	125 pairs	N = 140 was 8,092 tokens, just over the limit.
allam-2-7b context window	About 4,096 tokens	Limits it to about 70 pairs, regardless of the per-minute limit.
Calls per minute	About 1	Each request uses most of the minute's budget, so the app waits about 60 seconds between calls. One 5-call run takes 2 to 5 minutes.

Why UUIDs are expensive

Random UUIDs split into many small tokens. We measured about 1.7 characters per token, compared with roughly 4 for normal English. One key-value pair costs about 50 tokens, so 100 pairs is already about 5,400 tokens.

How the app handles the limits

- It estimates each request's tokens before running, with a 10% safety margin, and refuses to start if one request would exceed the limit. It also shows the largest N that fits.
- It tracks tokens used in the last 60 seconds and waits with a live countdown before each call.
- It shows Groq's 413 and context-length errors directly instead of retrying them.

Other limitations

- **One trial per position.** With T = 1, each result is a single answer. A 100% or 0% result is clear, but small differences between positions would need more trials, which the rate limit makes slow (T = 3 at N = 75 takes about 15 minutes).
- **Few models available.** Groq's free tier offered only three usable text models. The model named in our first plan, llama-3.1-8b-instant, had been removed.
- **Our hard mode is our own addition.** Look-alike keys are not part of the paper's experiment. They make the task harder but are not a direct comparison.
- **Windows setup.** Smart App Control on the test laptop blocked the compiled files of new pandas and pyarrow versions, so the app was rebuilt to run without them. This did not affect the results.

Conclusion

Within about 7,300 tokens of context, current models on Groq did not get lost in the middle of the paper's key-value task, even with look-alike distractors. The only failing model failed at every position, for a different reason. Testing whether the U-curve returns at longer contexts would need a paid tier or another provider with a higher token limit, plus more trials per position.